

10 Adressage dispersé et Table de hachage (*Hashtable*)

Les tables de hachage (*hashtable*) permettent d'effectuer les opérations d'insertion, d'enlèvement et de recherche en temps quasi constant, soit $O(1)$. Il s'agit d'une amélioration significative par rapport aux arbres binaires de recherche (Section 8.6) qui effectuent ces opérations en temps $O(\log n)$. Pour cette raison, les tables de hachages sont très utilisées pour implémenter des structures de données de type dictionnaire (*map*) contenant un très grand nombre d'entrées. Par exemple, les serveurs de noms de domaine (DNS) peuvent utiliser des tables de hachage afin de traduire efficacement des noms de domaine en adresses IP.

10.1 Motivation

Comment cherchez-vous un mot dans un dictionnaire ? Ou le nom d'une personne dans un bottin téléphonique ? Après avoir étudié les arbres binaires de recherche et la recherche dichotomique dans un tableau trié, vous pourriez être tentés par une recherche dichotomique. Cette technique consiste à couper l'intervalle de recherche par deux à chaque étape. Par exemple, si vous cherchez le mot « structure » dans un dictionnaire de 1024 pages, vous allez commencer par ouvrir le dictionnaire au milieu, soit vers la page 512. En vous apercevant que le mot « structure » vient après la page 512, vous allez encore couper par deux afin de limiter la recherche entre les pages 512 et 768. Cela se répétera jusqu'à ce que le mot soit trouvé. Dans le pire cas, $\log_2(1024) = 10$ pages auront été visitées.

Est-il possible de faire mieux ? Bien sûr que oui ! Comme le mot « structure » commence par la lettre « S », soit la 19^e lettre de l'alphabet, vous pourriez présumer que ce mot devrait se trouver autour du $19/26^e$ du dictionnaire. Donc, pourquoi ne pas tenter d'aller approximativement vers la page $19 \times 1024/26 = 748$? Une fois cette page ouverte, on avance ou recule jusqu'au bon mot. Évidemment, comme les mots ne sont pas distribués uniformément (davantage de mots commencent par S que par Z), l'expérience aide à estimer la bonne page.

Cette dernière méthode de recherche s'apparente à un **accès direct** dans une table. Fondamentalement, cela revient à trouver à quelle **adresse** (indice) une **clé** est stockée dans une table. Dans les prochaines sections, nous verrons comment réaliser ce type d'accès direct à l'aide d'une table de hachage.

10.2 Accès direct et adressage

Pour trouver en temps constant un élément précis dans un ensemble, il faut être capable de calculer en temps constant une adresse mémoire à partir de sa clé. Cela revient à calculer le **nombre ordinal** (position ou indice) d'un élément dans son ensemble. Pour certains types de données, comme les entiers, cela peut se faire naturellement.

Par exemple, imaginez que vous deviez développer un logiciel pour un club vidéo dans votre quartier. Ce logiciel doit permettre la recherche de fiches de client à partir d'une clé, soit le numéro de téléphone. Pour récupérer la fiche client en temps constant, on peut créer un tableau de la dimension du nombre de numéros de téléphones possibles. Dans le cas de numéros à 7 chiffres, la taille serait de 10^7 , soit 10 millions. Le code suivant illustre un exemple avec un tableau de pointeurs.

```

1 class FicheClient{
2  //...
3 };
4 class Base{
5 public:
6     Base(){
7         clients = new FicheClient*[10000000];
8     }
9     //...
10    FicheClient* getClient(int numero) {return clients[numero];}
11    void ajouterClient(int numero, FicheClient* c){clients[numero]=c;}
12    //...
13 private:
14    FicheClient** clients;
15 };

```

Il n'y a pas que pour les entiers que l'accès direct est possible. Théoriquement, il est possible de le faire pour n'importe quel type de données. Il suffit d'avoir une **relation d'ordre** totale et une fonction pour calculer le **nombre ordinal** (la position) de chaque élément. Par exemple, imaginons des noms d'utilisateur formés de huit lettres minuscules. L'adresse a d'un nom d'utilisateur $n = (l_0, l_1, \dots, l_7)$ peut être calculée à l'aide de l'équation 26. Les lettres sont représentées par des nombres de 0 à 25, c'est-à-dire $l_i \in \{0, 1, \dots, 25\}$.

$$a = \sum_{i=0}^7 26^i \cdot l_i \quad (26)$$

En pratique, les ensembles à mémoriser ont généralement une taille nettement inférieure à la taille du domaine des valeurs possibles. Par exemple, bien qu'il y ait 10 millions de numéros à 7 chiffres, notre club vidéo local n'a peut-être que quelques centaines de clients. Cela est encore plus vrai pour une base de noms d'utilisateur qui contient typiquement entre quelques usagers et quelques millions d'utilisateurs. Pourtant, le domaine des noms d'utilisateur à huit lettres contient $26^8 = 208827064576$ possibilités ! Avec des pointeurs de 64 bits, il faudrait une mémoire de 1.67×10^{12} octets (ou 1555 Go). Ainsi, l'accès direct avec un tel adressage est généralement non souhaitable, ou même irréalisable, car elle nécessite une trop grande quantité de mémoire.

10.3 Adressage réduit

Pour réduire l'utilisation de mémoire, il est possible de réduire la taille de l'espace d'adressage. Ainsi, au lieu d'utiliser un immense tableau pour stocker les valeurs d'un ensemble, on utilise un tableau de plus petite taille. La taille peut avoir le même ordre de grandeur que l'ensemble de valeurs à stocker. Pour savoir à quelle adresse (position) dans le tableau une clé est associée, une **fonction de réduction** est utilisée. Cette dernière utilise une **fraction de la clé** pour calculer une adresse. Généralement, cela se fait à l'aide d'un **modulo** ou d'une **division** arithmétique.

Pour illustrer la réduction d'adressage, reprenons l'exemple de notre club vidéo. Supposons qu'il a 1000 clients. Comment pourrions-nous réduire la taille du tableau requis pour stocker les pointeurs vers les fiches ? Une technique simple est de tronquer la clé en n'utilisant que les 4 derniers chiffres du numéro

de téléphone. Ainsi, au lieu d'avoir 10 millions d'entrées, il n'y en aurait que 10 milles. Une nette amélioration ! Cela revient à calculer le modulo 10000 du numéro de téléphone.

Cette technique fonctionnait plutôt bien à une époque où les numéros de téléphones d'un même quartier avaient de longs préfixes communs, où seuls les 3 ou 4 derniers chiffres changeaient. Cependant, rien ne garantit que chaque client possède un suffixe à 4 chiffres unique. Par exemple, si deux clients ont respectivement les numéros 222-4545 et 333-4545, la fonction de réduction ne retournera pas une adresse unique ! Cela provoque une **collision**. Une **collision** survient lorsque deux clés différentes génèrent la même adresse. Nous verrons à la Section 10.6 comment gérer ces collisions.

Pour l'exemple des noms d'utilisateur à huit lettres, nous pourrions être tentés de reprendre la même astuce, soit d'utiliser les trois premières ou dernières lettres pour calculer une adresse réduite. Ainsi, la taille du nouvel espace d'adressage sera réduit à $26^3 = 17576$ entrées. Toutefois, utiliser les 3 premières ou dernières lettres n'est généralement pas une bonne idée, car les noms d'utilisateur peuvent être mal distribués dans l'espace des possibilités. Comme les noms d'utilisateur sont souvent basés sur les noms et prénoms des personnes physiques, il risque d'y avoir beaucoup de collisions puisque beaucoup de noms auront des préfixes ou suffixes communs.

10.4 Adressage dispersé

L'utilisation d'une fraction de la clé a tendance à générer beaucoup de collisions lorsque la répartition des éléments n'est pas uniforme. Pour réduire le nombre de collisions potentielles, on utilise plutôt un **adressage dispersé**. Une **fonction de hachage** calcule une **valeur de hachage** à partir de la clé. Cette valeur sera utilisée comme **adresse dispersée**.

Idéalement, une fonction de hachage doit être le plus **chaotique** possible. Une fonction **chaotique** est une fonction dont une très petite variation de l'entrée entraîne une très grande variation de sa sortie. Cette propriété permet de réduire le taux de collisions entre les clés similaires. De plus, il est préférable que la fonction de hachage utilise toute la clé plutôt qu'une fraction de celle-ci.

La conception d'une bonne fonction de hachage dépend de la distribution des éléments. Dans l'exemple club vidéo, si tous les chiffres des numéros de téléphones sont relativement bien distribués, on peut se contenter d'utiliser directement le numéro de téléphone. Sinon, on peut « brasser » les chiffres pour tenter d'éviter les collisions. Pour ce faire, on peut multiplier les chiffres à l'aide de nombres premiers afin d'éviter que des motifs dans les clés soient reproduits dans les adresses. L'équation 27 présente un exemple de fonction de hachage pour convertir un numéro de téléphone n en une adresse dispersée.

$$h = 11 \cdot \lfloor n/10000 \rfloor + n \bmod 10000 \quad (27)$$

Les chaînes de caractères sont converties de façon similaire. Par exemple, en Java, la valeur de hachage h d'une `String` (chaîne de caractères) $\langle s_0, s_1, s_2, \dots, s_{n-1} \rangle$ est calculée de l'aide de l'équation 28.

$$h = \sum_{i=0}^7 s_i \cdot 31^{n-i-1} \quad (28)$$

10.5 Ébauche d'implémentation

Le code suivant présente une ébauche de déclaration d'une classe générique `EnsembleHache` pour représenter un ensemble d'éléments de type `T`. Pour l'instant, les collisions ne seront pas gérées. La représentation d'un `EnsembleHache` est un tableau linéaire de casiers (*buckets*). Un **casier** (*bucket*) contient un élément de type `T` et un booléen `occupe` indiquant son état d'occupation. L'interface publique de la classe `EnsembleHache` contient trois opérations : l'ajout d'un élément, l'enlèvement d'un élément et la vérification de l'existence d'un élément dans l'ensemble.

```

1  template <class T>
2  class EnsembleHache{
3      private:
4          class Casier{
5              public:
6                  Casier() : occupe(false){}
7                  T element;
8                  bool occupe;
9          };
10     Tableau<Casier> casiers;
11     public:
12     EnsembleHache(int taille=100):casiers(taille){}
13
14     void ajouter(const T&);
15     bool contient(const T&) const;
16     void enlever(const T&);
17 };

```

Le constructeur `EnsembleHache::EnsembleHache(int taille)` crée un tableau de casiers. La `taille` est le nombre de casiers à créer. Généralement, plus la taille est grande, moins il y aura de collisions. Il y a donc un compromis entre l'espace mémoire utilisé et le taux de collisions. Chaque casier est initialisé à l'aide du constructeur sans argument de `EnsembleHache::EnsembleHache::Casier`. Ce dernier initialise le membre `occupe` à `false` afin de marquer que le *casier* est initialement inoccupé.

La fonction `ajouter` est présentée ci-dessous et fonctionne comme suit. Une adresse dispersée est calculée à l'aide de la fonction de hachage et est placée dans la variable `h`. Ensuite, un modulo arithmétique est appliqué afin de réduire cette adresse à l'espace disponible, soit la taille du tableau. Une fois cette nouvelle adresse calculée, on s'assure que le casier soit bien inoccupé. S'il est déjà occupé, il y a collision. Comme les collisions ne sont pas encore gérées, on se contente de générer une erreur pour l'instant. Enfin, l'élément est inséré à son endroit et le casier est marqué occupé.

```

1  template <class T>
2  void EnsembleHache::ajouter(const T& element){
3      int h = element.valeurHachee();
4      int i = h % casiers.taille();
5      assert !casiers[i].occupe;
6      casiers[i].element = element;
7      casiers[i].occupe = true;
8  }

```

La fonction `contient` vérifie la présence d'un élément dans l'ensemble. On commence par calculer l'adresse dispersée `h` et l'adresse réduite `i`. L'élément existe si et seulement si le casier à la position `i` est occupé et contient l'élément à vérifier. À noter qu'il est très important de tester s'il s'agit du même élément puisque deux éléments différents peuvent avoir la même adresse réduite !

```

1 template <class T>
2 bool EnsembleHache::contient(const T& element) const {
3     int h = element.valeurHachee();
4     int i = h % casiers.taille();
5     return casiers[i].occupe && casiers[i].element==element;
6 }

```

La fonction `enlever` ci-dessous calcule une adresse dispersée `h` et une adresse réduite `i`. Optionnellement, selon le comportement souhaité, on peut vérifier si l'élément à supprimer est bien présent dans l'ensemble. Cela aurait pour effet d'empêcher le programmeur d'enlever un élément inexistant. Il s'agit d'un choix d'implémentation non recommandé. Enfin, on marque le casier `i` comme étant inoccupé.

```

1 template <class T>
2 void EnsembleHache::enlever(const T& element){
3     int h = element.valeurHachee();
4     int i = h % casiers.taille();
5     //assert casiers[i].occupe && casiers[i].element==element;
6     casiers[i].occupe = false;
7 }

```

La Figure 52 montre un exemple d'ajouts de A, H, N et V dans un `EnsembleHache`. Ici, la fonction de hachage retourne tout simplement la position de la lettre dans l'alphabet ordonné.

0	1	2	3	4	5	6	7	8	9	10
A		N					H			V

FIGURE 52 – Exemple d'une table de hachage

10.6 Gestion des collisions

Bien que nous puissions espérer qu'une bonne fonction de hachage génère peu de collisions, il est pratiquement impossible de les éviter puisque l'espace d'adressage réduit est nettement inférieur à la taille du domaine des clés possibles.

De plus, même si le nombre de casiers dépasse largement le nombre d'éléments à mémoriser, les collisions demeurent difficiles à éviter. Ce constat peut être illustré à l'aide du paradoxe des dates de naissances dans un groupe de n personnes. Quelle est la probabilité qu'au moins deux personnes d'un groupe de 25 aient la même date d'anniversaire ? On pourrait penser que c'est très peu puisque $25 \ll 365$. La réalité est qu'il y a 56.8 % de chances qu'au moins 2 personnes aient la même date d'anniversaire. Et ce nombre dépasse 99 % à partir de 60 personnes ! Cet exemple devrait vous convaincre que les collisions

sont pratiquement inévitables. Heureusement, il existe plusieurs stratégies pour gérer les collisions. Les plus fréquentes sont présentées ici.

10.6.1 Adressage ouvert

L'adressage ouvert consiste à résoudre une collision en utilisant une autre entrée (adresse alternative). L'adresse alternative peut être calculée de façon linéaire, quadratique ou à l'aide d'une seconde fonction de hachage.

Deux méthodes de résolution seront illustrées à l'aide d'un exemple qui consiste à ajouter les lettres suivante dans un `EnsembleHash` : A, N, V, L, W, H et K. La fonction de hachage utilisée retourne la position de la lettre dans l'alphabet. Les adresses dispersées sont : A(0), N(13), V(21), L(11), W(22), H(7) et K(10).

La Figure 53 illustre les étapes d'insertion à l'aide d'une stratégie ouverte avec résolution **linéaire**. Puisque le tableau contient 11 casiers, on utilise la fonction de réduction *modulo*11. On obtient ainsi les adresses réduites suivantes : A(0), N(2), V(10), L(0), W(0), H(7) et K(10). Le résultat à la fin de chaque étape est présenté sur une ligne. Les casiers grisés sont ceux inoccupés. Ceux en rouges indiquent les tentatives d'insertion suite à une collision. La case verte indique le casier où l'élément a été inséré. Les lettres A, N et V sont respectivement insérées dans les casiers $0 \bmod 11 = 0$, $13 \bmod 11 = 2$ et $21 \bmod 11 = 10$. La première collision survient à l'insertion de L, car le casier 0 contient déjà A. Avec la résolution linéaire, on place L dans le prochain casier disponible (1). De façon similaire, W ne peut être inséré dans le casier 0. Il faut alors le placer dans le premier disponible, soit le casier 3. La lettre H va directement dans le casier 7. Enfin, la lettre K est placée dans le casier 4 après avoir fait des tentatives dans les casiers 10, 0, 1, 2 et 3.

0	1	2	3	4	5	6	7	8	9	10
A										
A		N								
A		N								V
A	L	N								V
A	L	N	W							V
A	L	N	W				H			V
A	L	N	W	K			H			V

FIGURE 53 – Exemple avec une gestion de collision linéaire

La Figure 54 illustre le résultat d'une résolution **quadratique**. Avec cette résolution, une équation quadratique est utilisée pour déterminer les tentatives. Ici, on utilise la formule i^2 où i est représente le numéro de tentative, soit : 0, 1, 4, 9, 16, ... La première différence avec la résolution linéaire survient à l'insertion de W. Après l'échec dans les casiers 0 et 1, la lettre W est placée dans le casier 4. La résolution quadratique a pour avantage d'être davantage aux fonctions de hachages imparfaites qui généreraient des valeurs concentrées dans certaines régions de l'espace d'adressage.

0	1	2	3	4	5	6	7	8	9	10
A										
A		N								
A		N								V
A	L	N								V
A	L	N		W						V
A	L	N		W			H			V
A	L	N	K	W			H			V

FIGURE 54 – Exemple avec une gestion de collision quadratique

Une autre technique de résolution consiste à utiliser une deuxième fonction de hachage. Toutefois, celle-ci à elle seule n'est pas suffisante. Elle doit donc être combinée à une autre technique de résolution, comme les résolutions linéaires et quadratiques.

10.6.2 Chaînage à l'aide d'une deuxième structure

Une autre stratégie de gestion de collisions consiste à utiliser une deuxième structure pour stocker plusieurs objets dans une même entrée (adresse réduite). La Figure 55 illustre un exemple avec une structure linéaire telle une liste chaînée ou un tableau.

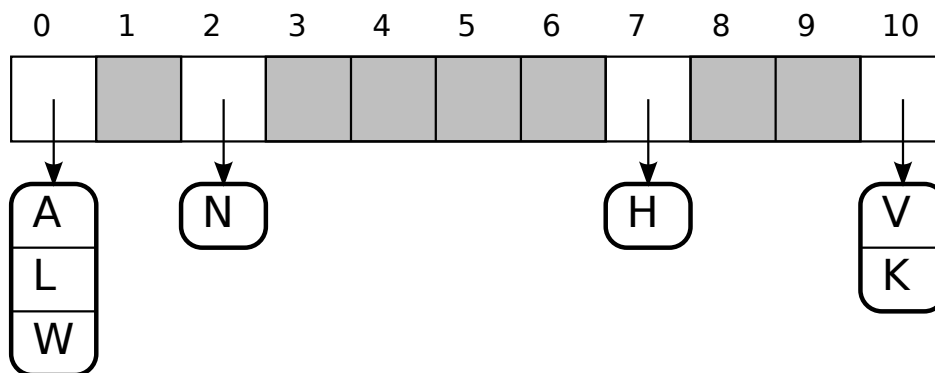


FIGURE 55 – Exemple d'une table de hachage avec chaînage

Le code suivant révisé la représentation de la classe `EnsembleHache::Casier`.

```

1  template <class T>
2  class EnsembleHache{
3      private:
4          class Casier{
5              public:
6                  Liste<T> entrees;
7          };
8          Tableau<Casier> casiers;
9      public:
10 //...
```

La Figure 56 illustre un exemple avec un arbre AVL.

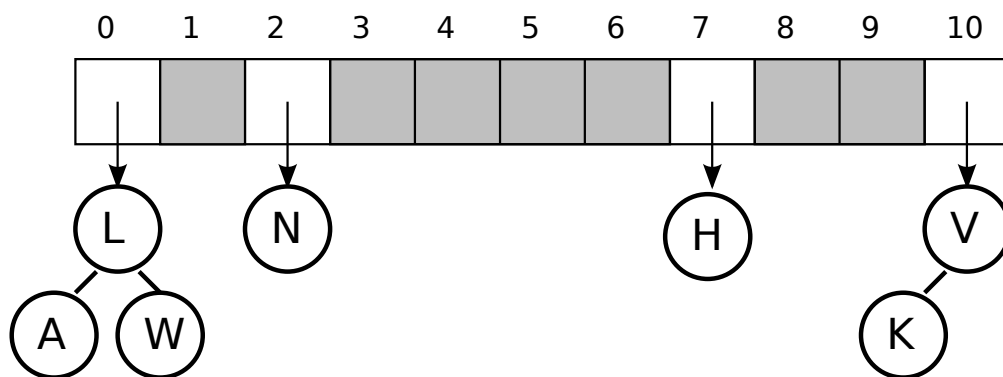


FIGURE 56 – Exemple d’une table de hachage avec arbres binaires

Le code suivant révisé la représentation de la classe `EnsembleHache::Casier`. Le code des fonctions est laissé en exercice.

```

1  template <class T>
2  class EnsembleHache{
3      private:
4          class Casier{
5              public:
6                  ArbreAVL<T> entrees;
7          };
8          Tableau<Casier> casiers;
9      public:
10 //...
```

10.7 Augmentation de la taille variable du tableau

La performance d’une table de hachage décroît en fonction du nombre de collisions. Le nombre de collisions dépend du **taux d’occupation**, soit le rapport nombre d’éléments sur la taille du tableau (nombre de casiers disponibles). Ce taux est aussi appelé **facteur de chargement** (*load factor*). Comme on ne

connaît pas toujours le nombre d'éléments à stocker à l'avance, une technique fréquente consiste à commencer avec un tableau de petite taille et à l'augmenter à chaque fois qu'une limite sur le facteur de chargement est atteinte. Pour éviter de redimensionner le tableau trop souvent, on peut réutiliser la même stratégie que pour les tableaux génériques, qui consiste à doubler la taille du tableau à chaque fois.

10.8 Table de hachage associative (dictionnaire)

Il est possible de créer une table de hachage associative afin d'associer des valeurs à des clés. Le code suivant donne l'ébauche de la déclaration d'une classe `DictionnaireHache`.

```
1 template <class K, V>
2 class DictionnaireHache{
3     private:
4         class Casier{
5             public:
6                 ListeMap<K, V> entrees;
7         };
8         Tableau<Casier> casiers;
9     public:
10    //...
```

10.9 Analyse et discussion

En pratique, l'adressage dispersé offre généralement de bonnes performances. Les opérations se font pratiquement en $O(1)$. Toutefois, une performance d'accès en temps $O(1)$ n'est pas toujours garantie. Dans les faits, la performance d'une table de hachage dépend de la fonction de hachage utilisée. Comme les fonctions de hachage sont généralement conçues de façon empirique, elle n'offrent pas de garanties théoriques pour disperser uniformément les éléments. Si une fonction s'avère peu chaotique, elle pourrait entraîner un haut taux de collisions. Dans le pire cas, la fonction de hachage placera tous les éléments dans un même casier. Ainsi, les gains de performance liés à l'adressage dispersés seront complètement annulés.

La performance dans le pire cas dépend de la gestion des collisions. Avec un adressage ouvert et avec un chaînage avec une liste chaînée, les opérations de recherche et d'enlèvement se feront en temps $O(n)$. Cela s'avère pire que $O(\log n)$ offert par les arbres binaires de recherche. Au mieux, avec une stratégie de chaînage avec un arbre binaire, la complexité temporelle sera de $O(\log n)$, et ce, même dans le pire cas.